

Grupa: 134 Numele și prenumele (cu MAJUSCULE): _____

Arhitectura Sistemelor de Calcul: Parțial - 10% din nota finală

Fie $x = \text{len}(\text{nume}) \times \text{len}(\text{prenume})$, considerați că este definit în secțiunea `.data` din Exercițiul 3 și Exercițiul 4

Exercițiul 1 1.5p Scrieți dimensiunea alocată în RAM, atât în *biți*, cât și în *Bytes*, pentru următoarele variabile declarate în secțiunea `.data`:

Declarare în <code>.data</code>	dimensiune în biți	dimensiune în Bytes
<code>str1: .ascii "Partial la ASC\n"</code>		
<code>myWord: .word 42</code>		
<code>myLong: .long 0xfde2025</code>		
<code>str2: .asciz "Restanta la ASC\n"</code>		
<code>myChar: .ascii '1'</code>		

Scrieți un apel de sistem `WRITE` (codul apelului este 4) pentru șirul de caractere `str2`, astfel încât să afișați, la `standard output`, întreg conținutul șirului de caractere și nimic altceva în plus.

Exercițiul 2 1p Completați următorul tabel, cu valorile corecte în bazele 2 și 16.

Baza 2	Baza 8
	7 0 1 5 3
101 011 110 100 000 111 010 111	

Exercițiul 3 2p Fie următorul program, scris în *assembly* (procesor Intel x86, sintaxa *AT&T*):

```
.data                                et1:
    z: .byte 0b10000                divl t
    t: .byte 128                     decl %eax
.text
.global main                        et_exit:
main:                                movl $1, %eax
    movl x, %eax                     xorl %ebx, %ebx
    mull z                           int $0x80
```

Executăm, în *debugger*, următoarele comenzi:

`b main; b et_exit; run; stepi; stepi; i r eax; i r edx; stepi; i r eax; i r edx;`

Valorile pe care le vom obține sunt, în ordine:

EAX (1)	EDX (1)	EAX (2)	EDX (2)

Important! Completați valorile în baza 16.

Exercițiul 4 1.5p Fie următoarele instrucțiuni în *main*:

```
movl x, %eax
addl $0x800001, %eax
movl $4096, %ecx
mul %ecx
divl p
```

Descrieți ce obținem dacă $p = 32$. Dar dacă $p = 4$?

Exercițiul 5 1p Fie următorul program, scris în *assembly* (procesor Intel x86, sintaxa AT&T):

```
.data                                xorl %edx, %edx                                jmp et_exit
    # x calculat anterior          divl y                                et2:
    y: .long 11                    addl %edx, %eax                                movl $2, %ebx
.text                                mul %eax                                et_exit:
.global main                        sub %edx, %eax                                mov $1, %eax
main:                               cmp $62, %eax                                xorl %ebx, %ebx
    movl x, %eax                   jl et2                                int $0x80
    addl y, %eax                   et1:                                movl $1, %ebx
    subl $7, %eax                  movl $1, %ebx
```

Completați valorile regiștrilor în momentul în care executarea programului ajunge în dreptul etichetei *et_exit* (după ce am rulat în *debugger* comenzile *b et_exit; run; i r*):

EAX	EBX	EDX

Exercițiul 6 1p Explicați dacă următoarele două secvențe de cod accesează elementul de pe indexul al treilea al unui array de *long-uri*, stocat în RAM la adresa cu numele *v*. Sunt secvențele de cod echivalente? Dacă da, de ce? Dacă nu, ce accesează fiecare, în parte?

a. `mov $v, %edi; movl $12, %ecx; movl (%edi, %ecx, 4), %eax`

b. `lea v, %edi; addl $12, %edi; xorl %ecx, %ecx; movl (%edi, %ecx, 4), %eax`

Exercițiul 7 2p Scrieți o secvență de cod în *main* care să calculeze pentru un array de întregi *v*, de lungime *n*, $\sum_{i=0}^{n-1} (-2)^i v[i]$. Aceste informații sunt deja în secțiunea *.data*. Soluția voastră trebuie să înceapă de la eticheta *main*.